Manual Técnico del Sistema de Gestión de Puentes

Arquitectura, módulos y operación

Equipo de desarrollo SGP

Universidad de San Carlos de Guatemala — Centro Universitario de Occidente

Octubre de 2026

Índice

1 Introducción y alcance **1**
1.1 Propósito . . . 1
1.2 Alcance funcional . . . 1
1.3 Límites . . . 1

2 Tecnologías y dependencias principales **1**

3 Requisitos de hardware y software **2**
3.1 Software . . . 2
3.2 Hardware . . . 3

4 Instalación y configuración del entorno **3**
4.1 1. Configurar variables . . . 3
4.2 2. Iniciar PostgreSQL/PostGIS . . . 3
4.3 3. Iniciar la API . . . 3
4.4 4. Iniciar Angular . . . 4
4.5 Alternativa con Compose . . . 4
4.6 Configuración de archivos . . . 4

5 Base de datos y relaciones principales **4**

6 Estructura del proyecto **6**
6.1 Backend . . . 6
6.2 Cliente . . . 6
6.3 Persistencia y documentación . . . 6

7 Estrategia y ejecución de pruebas **7**
7.1 Verificaciones de CI . . . 7

8 1. Arquitectura y modelo de datos **7**
8.1 Descripción . . . 8
8.2 Tecnologías . . . 8
8.3 Funcionamiento . . . 8
8.4 Diagrama Mermaid . . . 9

9 Seguridad y sesión **9**
9.1 Descripción . . . 9
9.2 Tecnologías . . . 9
9.3 Funcionamiento . . . 10
9.4 Diagrama Mermaid . . . 10

10 Usuarios y cursos **11**
10.1 Descripción . . . 11
10.2 Tecnologías . . . 11
10.3 Funcionamiento . . . 11
10.4 Diagrama Mermaid . . . 12

11 Puentes y asignaciones **13**

11.1 Descripción . . . 13
11.2 Tecnologías . . . 13
11.3 Funcionamiento . . . 13
11.4 Diagrama Mermaid . . . 14

12 Formularios y calibración **15**
12.1 Descripción . . . 15
12.2 Tecnologías . . . 15
12.3 Funcionamiento . . . 15
12.4 Diagrama Mermaid . . . 15

13 Índice de condición **16**
13.1 Descripción . . . 16
13.2 Tecnologías . . . 16
13.3 Funcionamiento . . . 16
13.4 Diagrama Mermaid . . . 17

14 Archivos y evidencias **18**
14.1 Descripción . . . 18
14.2 Tecnologías . . . 18
14.3 Funcionamiento . . . 18
14.4 Diagrama Mermaid . . . 19

15 Inspecciones **19**
15.1 Descripción . . . 19
15.2 Tecnologías . . . 19
15.3 Funcionamiento . . . 20
15.4 Diagrama Mermaid . . . 20

16 Offline y sincronización **21**
16.1 Descripción . . . 21
16.2 Tecnologías . . . 21
16.3 Funcionamiento . . . 21
16.4 Diagrama Mermaid . . . 22

17 Revisión académica **22**
17.1 Descripción . . . 22
17.2 Tecnologías . . . 22
17.3 Funcionamiento . . . 23
17.4 Diagrama Mermaid . . . 23

18 Mantenimiento **24**
18.1 Descripción . . . 24
18.2 Tecnologías . . . 24
18.3 Funcionamiento . . . 24
18.4 Diagrama Mermaid . . . 25

19 Foro y notificaciones **26**
19.1 Descripción . . . 26

19.2 Tecnologías 26
19.3 Funcionamiento 26
19.4 Diagrama Mermaid 27

20 Auditoría **27**
20.1 Descripción 27
20.2 Tecnologías 27
20.3 Funcionamiento 27
20.4 Diagrama Mermaid 28

21 Despliegue y recuperación **28**
21.1 Descripción 28
21.2 Tecnologías 29
21.3 Funcionamiento 29
21.4 Diagrama Mermaid 29

1 Introducción y alcance

1.1 Propósito

El Sistema de Gestión de Puentes (SGP) es una aplicación web progresiva para administrar el inventario de puentes y apoyar su inspección técnica, revisión académica, colaboración y mantenimiento. Está dirigido a administradores, catedráticos, estudiantes y profesionales externos, con permisos asociados al rol y al contexto de trabajo.

1.2 Alcance funcional

El sistema relaciona puentes y su ubicación territorial con formularios de inspección versionados, evidencias, resultados del índice de condición y acciones de mantenimiento. Para el trabajo de campo dispone de preparación de salidas y persistencia local que permite continuar capturas con conectividad limitada y sincronizarlas posteriormente.

La solución se compone de un cliente Angular PWA, una API REST Spring Boot y PostgreSQL con PostGIS. La API concentra validaciones y reglas de negocio; el cliente presenta las funciones y se comunica con ella. Los documentos generales describen el contexto, entorno y prácticas transversales. Los capítulos M01–M14 desarrollan arquitectura y dominios concretos.

1.3 Límites

El SGP registra información y apoya procesos técnicos y académicos. La publicación de un resultado o una propuesta de mantenimiento depende de las reglas configuradas y de las acciones autorizadas en el sistema. Este manual describe el comportamiento implementado; no establece criterios de ingeniería civil externos ni afirma que un despliegue particular esté activo.

2 Tecnologías y dependencias principales

La tabla resume las tecnologías presentes en la configuración del proyecto. Las versiones exactas se fijan en Maven, npm o las imágenes de contenedor según corresponda.

Capa	Tecnología	Función principal
Backend	Java 21, Spring Boot 3.3	API REST, lógica de negocio, validación y seguridad.
Persistencia	Spring Data JPA, Hibernate Spatial, MapStruct	Acceso ORM, consultas geográficas y mapeo entre DTO y entidades.
Contratos	Springdoc OpenAPI, JSON Schema Validator	Descripción de API y validación de esquemas dinámicos.
Seguridad	Spring Security, JJWT, BCrypt	Autorización, tokens y protección de credenciales.
Base de datos	PostgreSQL 16, PostGIS 3.5, Flyway	Datos relacionales y espaciales, con evolución versionada del esquema.
Cliente	Angular 21, TypeScript 5.9, RxJS 7	Aplicación web y coordinación de operaciones asíncronas.

Capa	Tecnología	Función principal
Interfaz y mapas	PrimeNG, PrimeIcons, MapLibre GL	Componentes de interfaz y visualización geográfica.
PWA y trabajo local	Angular Service Worker, Dexie, IndexedDB	Caché de recursos y persistencia local estructurada.
Archivos	AWS SDK for Java S3, Apache PDFBox, ImageIO/WebP	Almacenamiento de objetos, generación/validación de PDF y procesamiento de imágenes.
Operación	Docker Compose, Nginx, MinIO, ShedLock	Entorno contenedorizado, proxy, almacenamiento compatible con S3 y coordinación de tareas.
Pruebas	JUnit/Spring Test, Testcontainers, Playwright	Pruebas de backend, integración con PostgreSQL y pruebas de navegador.

El backend se compila con Maven. El cliente utiliza npm y el archivo de bloqueo para instalar las dependencias declaradas. En CI se usa Node.js 24; la imagen de compilación del cliente también utiliza Node 24.

3 Requisitos de hardware y software

3.1 Software

Componente	Requisito configurado	Observación
Backend local	JDK 21 y Maven	No hay Maven Wrapper; la imagen de compilación utiliza Maven 3.9.9.
Cliente local	Node.js compatible con Angular; Node 24 recomendado	CI y Docker usan Node 24. npm instala desde `package-lock.json`.
Base de datos local	Docker y Docker Compose, o PostgreSQL 16 con PostGIS 3.5	Compose publica PostgreSQL en `localhost:55432`.
Aplicación completa en contenedores	Docker con soporte de Compose	El cliente se sirve por Nginx en `localhost:8088`.
Pruebas de integración	Docker accesible al proceso de pruebas	Testcontainers levanta PostgreSQL/PostGIS para pruebas de integración.
Navegador	Navegador moderno con soporte para PWA, IndexedDB y service workers	Necesario para las funciones locales del cliente.

Para ejecutar exclusivamente el cliente, se requieren Node.js y npm. Para iniciar la API localmente

se necesita Java, Maven, una base PostgreSQL/PostGIS accesible y las variables requeridas. Docker simplifica la base local y las pruebas de integración.

3.2 Hardware

El proyecto no fija cantidades mínimas de CPU, memoria o almacenamiento para equipos de desarrollo o servidores. Los recursos necesarios dependen del volumen de datos, cantidad de usuarios, imágenes y procesos concurrentes. En desarrollo, el equipo debe disponer de recursos suficientes para las herramientas locales y los contenedores que se ejecuten simultáneamente.

4 Instalación y configuración del entorno

Esta guía configura el entorno local con la base de datos en Docker y la API y el cliente ejecutados localmente. Se requieren JDK 21, Maven, Node.js/npm y Docker Compose.

4.1 1. Configurar variables

Desde la raíz del proyecto, cree el archivo local de configuración del backend:

```
cp sgp-api/.env.example sgp-api/.env
```

Revise el archivo y establezca credenciales válidas para correo SMTP y administrador inicial. La plantilla configura PostgreSQL local y valores de desarrollo. Sustituya la clave JWT por una cadena aleatoria de al menos 32 caracteres. No comparta ni versiona el archivo `.env`.

Las variables esenciales para iniciar el backend incluyen URL, usuario y contraseña de la base; usuario y contraseña de migración; clave JWT; correo y contraseña SMTP; y credenciales para crear el administrador inicial. `MAIL_HOST`, `MAIL_PORT`, `MAIL_FROM`, CORS y otros parámetros tienen valores predeterminados o pueden ajustarse en el mismo archivo.

4.2 2. Iniciar PostgreSQL/PostGIS

En la raíz del proyecto:

```
docker compose up -d postgres
docker compose ps
```

Espere a que PostgreSQL aparezca saludable. Compose crea la base `sgp_db`, el usuario de aplicación y el usuario migrador. Los datos locales permanecen en el volumen `postgres_data`.

4.3 3. Iniciar la API

En una terminal:

```
cd sgp-api
mvn clean spring-boot:run
```

Flyway aplica las migraciones y Hibernate valida el esquema al iniciar. La API queda en `http://localhost:8090`; compruebe su estado en `/actuator/health`. La documentación OpenAPI está disponible cuando la configuración local permite exponerla.

4.4 4. Iniciar Angular

En otra terminal:

```
cd sgp-client
npm ci
npm start
```

Abra `http://localhost:4200`. El servidor de desarrollo reenvía las solicitudes `/api` hacia la API local, por lo que no requiere configurar una URL distinta del backend para este flujo.

4.5 Alternativa con Compose

Para construir y ejecutar los servicios de la aplicación definidos en Compose:

```
docker compose up -d --build
docker compose ps
```

La aplicación web queda disponible en `http://localhost:8088`. Para detener contenedores sin borrar datos use `docker compose down`; evite `docker compose down -v` si necesita conservar la base local.

4.6 Configuración de archivos

El backend usa almacenamiento local de forma predeterminada. La configuración también admite proveedor S3 con región y bucket, y endpoint explícito para MinIO con credenciales de desarrollo. Asegure que el bucket exista y sea privado cuando se use este proveedor. No se necesita MinIO para iniciar el flujo local predeterminado.

5 Base de datos y relaciones principales

La persistencia usa PostgreSQL con PostGIS. Flyway versiona las modificaciones del esquema y Hibernate valida las entidades contra la estructura existente; la aplicación no crea tablas automáticamente. Las relaciones centrales conectan usuarios y cursos con puentes, inspecciones, revisiones y resultados técnicos.


DEPARTAMENTO
uuid id PK
string nombre
contiene
MUNICIPIO
uuid id PK
uuid departamento_id FK
string nombre
ROL
uuid id PK
string nombre
asigna
ubica
USUARIO
uuid id PK
uuid rol_id FK
string email
PUENTE
uuid id PK
uuid municipio_id FK
string codigo
geography ubicacion
VERSION_FORMULARIO
uuid id PK
string codigo
jsonb esquema
ASIGNATURA
uuid id PK
string codigo
define
imparte
autor
inspeccionado
estructura
CURSO
uuid id PK
uuid asignatura_id FK
uuid catedratico_id FK
string periodo
participa
revisa
INSPECCION
uuid id PK
uuid puente_id FK
uuid autor_id FK
uuid version_formulario_id FK
jsonb datos
inscribe
enviada
calcula
asignado
CURSO_ESTUDIANTE
uuid id PK
uuid curso_id FK
uuid estudiante_id FK
REVISION
uuid id PK
uuid inspeccion_id FK
uuid revisor_id FK
int numero_ciclo
RESULTADO_IC
uuid id PK
uuid inspeccion_id FK
decimal indice_calculado
recibe
ASIGNACION_PUENTE
uuid id PK
uuid curso_estudiante_id FK
uuid puente_id FK

La relación académica entre cursos, estudiantes y puentes se representa con entidades de asociación, lo que permite conservar sección, estado de inscripción y revocación de asignaciones. Una inspección conserva referencias a su puente, autor y versión de formulario. Puede generar varios resultados IC porque se registran publicaciones y recálculos; los ciclos de revisión se distinguen por número. El diagrama omite catálogos, archivos, bitácoras e historiales para facilitar su lectura.

Los datos espaciales de puentes se almacenan con PostGIS. El contenido variable de formularios y algunos desgloses se guarda en JSONB, mientras las relaciones y restricciones de integridad se expresan con claves foráneas, índices y restricciones de base de datos.

6 Estructura del proyecto

El repositorio separa la aplicación en cliente, API, infraestructura y documentación.

Área	Organización	Responsabilidad
`sgp-api/`	Backend Spring Boot	API REST y lógica por dominios funcionales.
`sgp-client/`	Aplicación Angular PWA	Interfaz, navegación, acceso a API y experiencia offline.
`infra/`	PostgreSQL, RDS, staging, producción y respaldos	Inicialización y automatización de operación.
`docs/`	Arquitectura, pruebas, base de datos, API y manual técnico	Documentación del sistema y de sus procesos.
`.github/workflows/`	Flujos de CI y publicación	Validación automatizada y construcción de imágenes.

6.1 Backend

El código Java se organiza por dominios, entre ellos usuarios, cursos, puentes, formularios, inspecciones, revisión, mantenimiento, archivos, foro, notificaciones y sincronización. Dentro de cada dominio se separan según necesidad los controladores REST, DTO, entidades, repositorios, servicios, mapeadores y validaciones. `common/` reúne capacidades transversales como configuración, seguridad, errores y auditoría.

6.2 Cliente

La aplicación organiza pantallas y servicios por `features/`. `core/` aloja navegación y capacidades transversales; `offline/` encapsula la persistencia local y sincronización; `shared/` mantiene componentes reutilizables; y `layouts/` y `theme/` contienen la estructura y estilos comunes. Las rutas cargan funcionalidades de forma diferida cuando corresponde.

6.3 Persistencia y documentación

Las migraciones Flyway del backend describen la evolución de PostgreSQL/PostGIS. La documentación API, las guías de prueba y el diccionario de datos acompañan el código. El manual técnico agrupa capítulos generales y módulos funcionales para que puedan consultarse juntos o por separado.

7 Estrategia y ejecución de pruebas

La validación combina pruebas de lógica, contratos y seguridad con pruebas de integración y verificaciones de construcción. La presencia de una prueba no demuestra que se haya ejecutado ni que haya pasado; los resultados deben consultarse en la ejecución correspondiente.

Alcance	Herramienta y cobertura	Comando habitual
Backend	JUnit y Spring Test; pruebas de servicios, controladores y seguridad.	`cd sgp-api && mvn test`
Integración backend	Testcontainers con PostgreSQL/PostGIS para persistencia y flujos completos.	`cd sgp-api && mvn verify`
Estilo backend	Spotless comprueba formato Java.	`cd sgp-api && mvn spotless:check`
Cliente	Pruebas unitarias Angular configuradas por el builder de pruebas del proyecto.	`cd sgp-client && npm test`
PWA en navegador	Playwright para escenarios de campo, conectividad y sincronización.	`cd sgp-client && npm run test:pwa`
Calidad cliente	ESLint, Prettier y compilación Angular.	`npm run lint`, `npm run format:check`, `npm run build`

Las pruebas de integración de backend necesitan que Docker esté disponible para que Testcontainers levante la base temporal. La suite Playwright usa un servidor de pruebas propio y concentra los escenarios offline/PWA; algunas fases adicionales se habilitan con variables de entorno definidas por la configuración de pruebas.

Para habilitar una fase específica de HU-062, use el indicador correspondiente antes del comando, por ejemplo `HU062_FASE1=true npm run test:pwa`; también están definidos `HU062_FASE2`, `HU062_FASE3` y `HU062_PAQUETES`.

7.1 Verificaciones de CI

El flujo de integración continua ejecuta `mvn -B spotless:check verify` para el backend. Para el cliente instala dependencias desde el archivo de bloqueo y ejecuta lint, formato y build. La configuración del flujo no ejecuta los comandos de pruebas del cliente ni la suite Playwright; deben lanzarse aparte cuando se valide ese alcance.

Al documentar un resultado de prueba, indique el comando, el entorno y su salida real. No infiera aprobación a partir de que el test esté presente o la compilación se haya completado.

8 1. Arquitectura y modelo de datos

8.1 Descripción

El SGP administra el inventario, las inspecciones y el mantenimiento de puentes mediante una aplicación cliente-servidor. El cliente web Angular consume una API REST Spring Boot. PostgreSQL conserva los datos del sistema; PostGIS añade soporte para ubicaciones y consultas geográficas.

8.2 Tecnologías

Componente	Tecnologías relevantes
Cliente web	Angular 21, TypeScript, RxJS y Angular Service Worker. Dexie sobre IndexedDB para datos locales offline.
API	Java 21, Spring Boot 3.3, Spring Web, Spring Security, Spring Data JPA, Spring Validation y MapStruct. Springdoc publica el contrato OpenAPI.
Persistencia	PostgreSQL 16, PostGIS y Flyway para las migraciones versionadas. Hibernate accede a los datos y valida su esquema.
Publicación y archivos	Nginx sirve el cliente y actúa como proxy; Docker Compose integra el entorno local. La API admite almacenamiento local o S3 compatible, incluido MinIO en desarrollo.

8.3 Funcionamiento

El navegador recibe la aplicación Angular desde Nginx. Las pantallas delegan las operaciones a servicios del cliente, que llaman por HTTP los endpoints funcionales bajo `/api/v1`. En el entorno desplegado, Nginx sirve la aplicación y reenvía las solicitudes `/api/` a la API; en desarrollo, el proxy de Angular las dirige al backend local.

La API aplica seguridad y validación en la frontera HTTP, delega las reglas a los servicios de dominio y accede a PostgreSQL mediante repositorios JPA o consultas SQL específicas. Los controladores intercambian DTO con el cliente; las entidades JPA permanecen dentro del backend. Los errores se devuelven en formato `ProblemDetail`, con código y detalles de validación cuando corresponde.

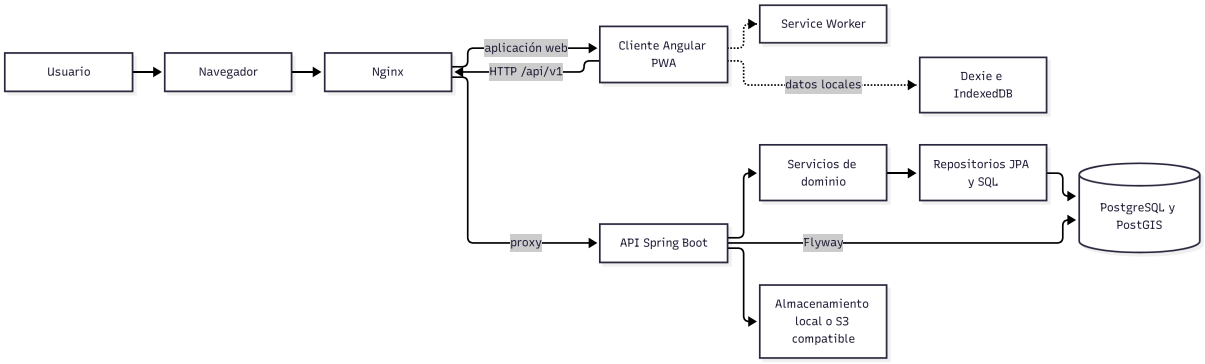
El backend se organiza por dominios funcionales, entre ellos usuarios, cursos, puentes, formularios, inspecciones, revisiones y mantenimiento. Cada dominio contiene las capas que necesita; `common` agrupa capacidades compartidas. El frontend se organiza en funcionalidades, con servicios para las llamadas HTTP y rutas que permiten cargar cada sección de manera diferida. `core` concentra infraestructura compartida y `offline` encapsula el almacenamiento local.

Los datos del servidor se modelan principalmente en tablas relacionales. Las entidades centrales vinculan usuarios, cursos e inscripciones con puentes y asignaciones; los puentes con inspecciones; las inspecciones con versiones de formulario, revisiones y resultados del índice de condición; y las órdenes de mantenimiento con planes, ejecuciones y evidencias. Foro, notificaciones, auditoría e idempotencia respaldan varios flujos.

El esquema también utiliza JSONB para esquemas, respuestas y configuraciones versionadas; tipos geográficos para ubicaciones y límites territoriales; e índices y restricciones para consultas e invariantes de datos. Los archivos binarios se guardan mediante el proveedor configurado, mientras la base de datos conserva sus registros y relaciones.

Flyway aplica las migraciones SQL del proyecto, actualmente desde V1 hasta V25. El navegador también puede conservar recursos de la PWA, algunos datos de referencia y datos de trabajo en IndexedDB. La sincronización offline se comunica con el backend mediante la API; IndexedDB no sustituye a la base de datos del servidor.

8.4 Diagrama Mermaid



Consideraciones técnicas

- Flyway gobierna la evolución del esquema. Hibernate tiene `ddl-auto=validate`: comprueba que el mapeo concuerde con la base, pero no crea ni actualiza tablas.
- Los cambios estructurales requieren nuevas migraciones; las ya aplicadas no se modifican.
- JSONB permite conservar estructuras variables, pero las solicitudes siguen sujetas a validaciones de dominio y esquema.
- Las ubicaciones y consultas espaciales dependen de PostGIS.
- Los perfiles y variables de ambiente determinan conexiones, seguridad y proveedor de archivos. La configuración disponible no confirma por sí sola qué servicios externos están desplegados.

9 Seguridad y sesión

9.1 Descripción

Este módulo controla el acceso a la API y mantiene la sesión entre el navegador y el servidor. Incluye registro verificado, autenticación, segundo factor, renovación de credenciales y autorización por roles.

9.2 Tecnologías

- Spring Security y JWT con JJWT.
- BCrypt para contraseñas y OTP de un solo uso.
- Angular, RxJS, guards e interceptores HTTP.

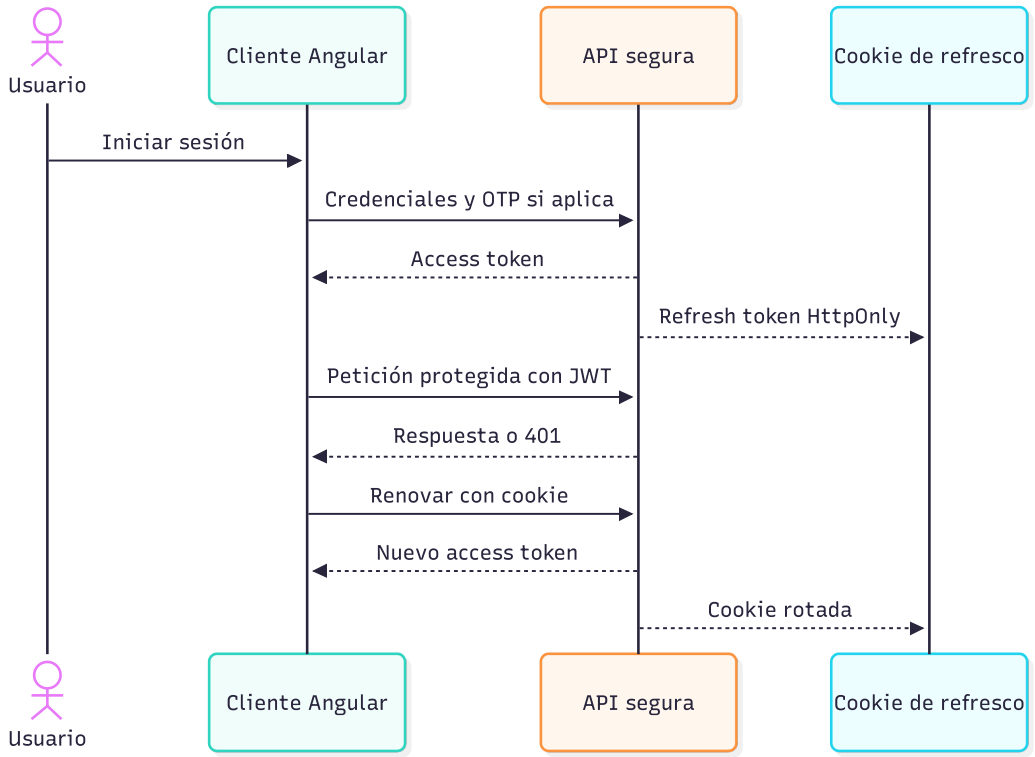
9.3 Funcionamiento

El registro de estudiantes requiere verificar un código enviado por correo; la cuenta conserva además su estado de activación académica. En el inicio de sesión, la API comprueba la contraseña y el estado de la cuenta. Si el segundo factor está habilitado, solicita un OTP antes de completar la sesión.

La API emite un access token JWT de vida limitada y un refresh token aleatorio. El access token se conserva en memoria en el cliente; el refresh token se entrega en una cookie HttpOnly. La renovación rota el refresh token y revoca el anterior. Ante una respuesta 401, el interceptor intenta renovar la sesión y repetir una vez la solicitud original.

En cada petición protegida, Spring Security valida el JWT y contrasta el estado actual, rol y versión de tokens del usuario. Los guards ayudan a controlar la navegación, pero la autorización efectiva reside en el servidor. La recuperación de contraseña valida un OTP, cambia la contraseña y revoca las sesiones activas.

9.4 Diagrama Mermaid



Consideraciones técnicas

- La cookie de refresco usa atributos configurables; `Secure` depende del entorno y `SameSite` tiene un valor predeterminado restrictivo.
- La caché local del perfil permite mostrar contexto durante una desconexión, pero no valida una sesión ante la API.
- Los OTP expiran, limitan intentos y se consumen una sola vez; las rutas públicas y los límites de solicitudes son configurados por el backend.

10 Usuarios y cursos

10.1 Descripción

Este módulo administra las cuentas y los vínculos académicos o profesionales que habilitan la participación en inspecciones y otras tareas del sistema.

10.2 Tecnologías

- Spring Boot, Spring Data JPA y Bean Validation.
- PostgreSQL y Flyway.
- Angular y formularios reactivos.

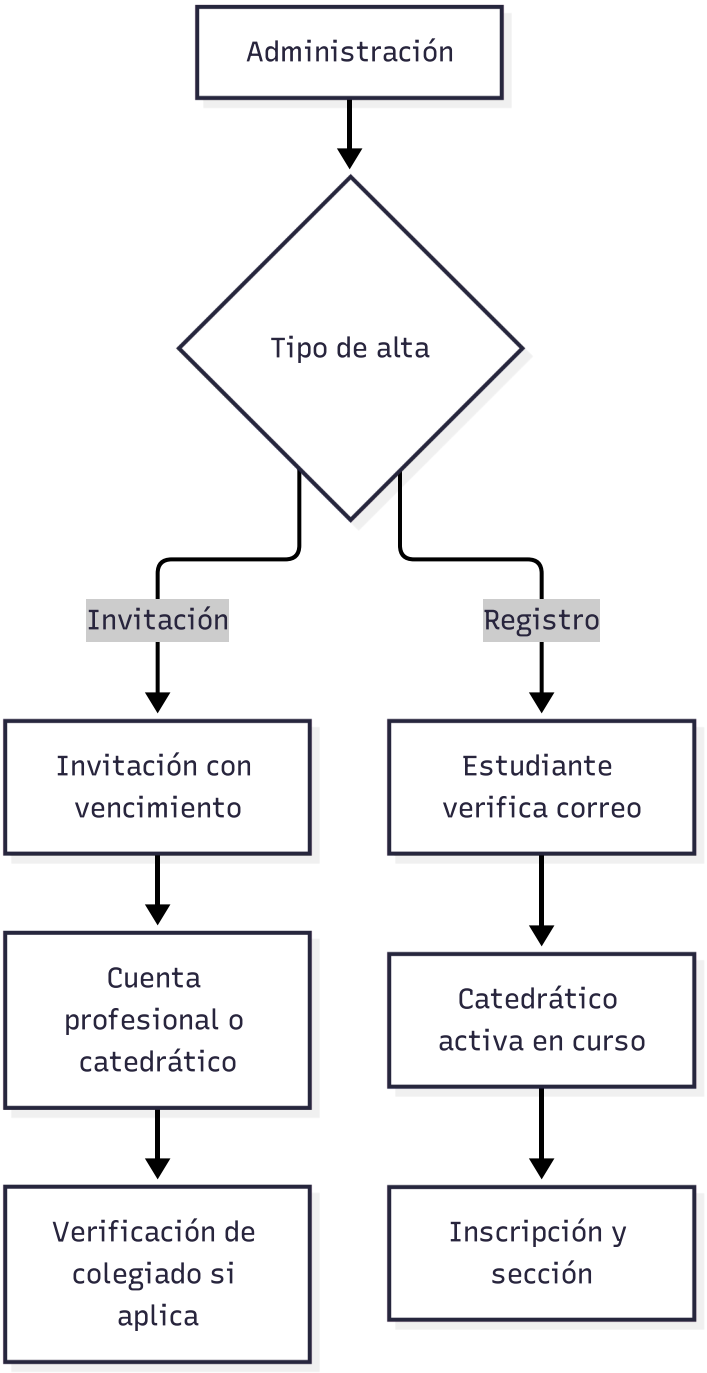
10.3 Funcionamiento

Un administrador puede invitar a usuarios con perfil de catedrático o profesional externo. La invitación se asocia a un token aleatorio con vencimiento; el destinatario valida el enlace y completa sus credenciales. Los profesionales registran su número de colegiado, que debe ser verificado por administración antes de habilitar las operaciones que lo requieren.

Los estudiantes se registran y verifican su correo, pero permanecen pendientes de activación. Un catedrático puede activar a un estudiante dentro de un curso activo que tenga asignado, indicando sección. La operación valida que el estudiante esté pendiente y no tenga ya una inscripción activa, y crea su inscripción académica.

Los cursos relacionan período, asignatura y catedrático. Administración puede crearlos, asignar o cambiar responsables y finalizarlos; las inscripciones vinculan estudiantes activos con curso y sección.

10.4 Diagrama Mermaid



Consideraciones técnicas

- La invitación no equivale a una sesión activa: el usuario debe completar el proceso de aceptación y establecer sus credenciales.
- La activación estudiantil depende de un curso vigente y de la asignación del catedrático; los roles y estados de cuenta se explican en el módulo de seguridad.
- La vigencia de inscripciones y colegiaturas es relevante para determinar qué operaciones académicas puede realizar cada usuario.

11 Puentes y asignaciones

11.1 Descripción

Este módulo mantiene el inventario de puentes, valida su ubicación y controla las solicitudes de alta y las asignaciones de trabajo a cursos.

11.2 Tecnologías

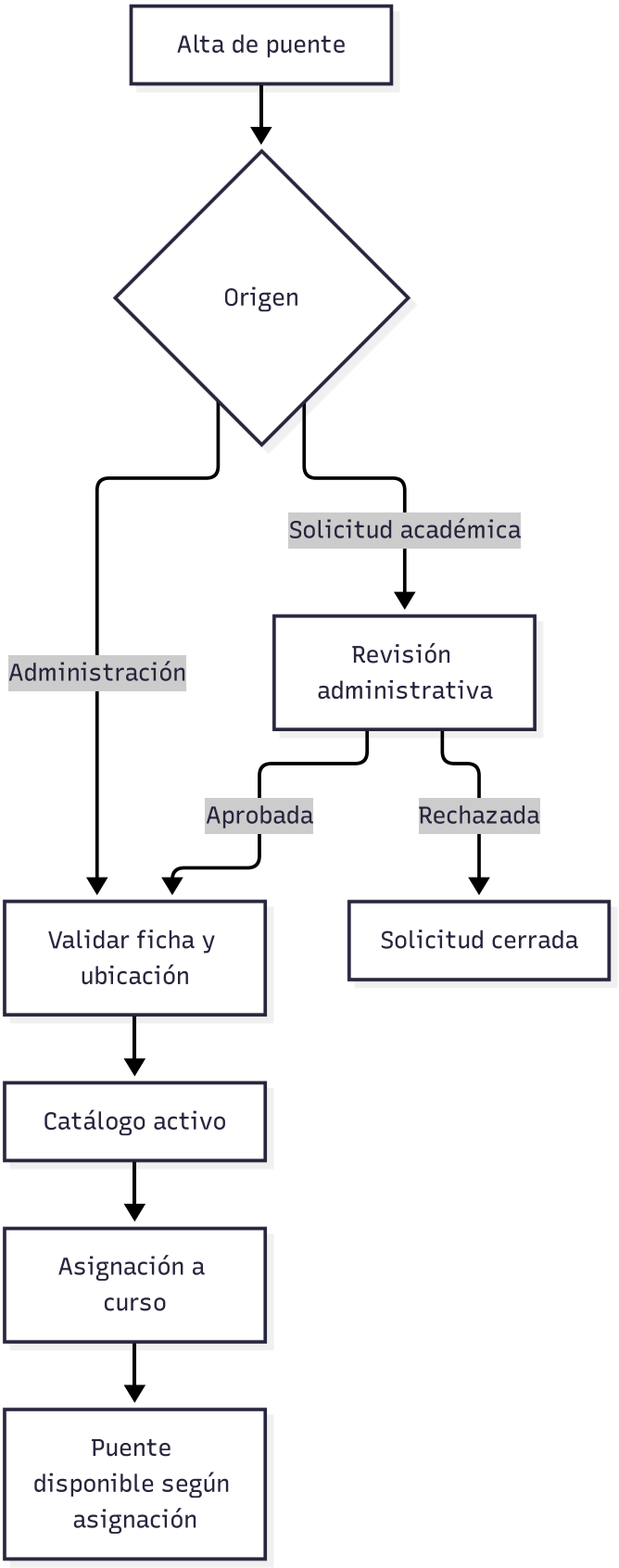
- Spring Boot, Spring Data JPA e Hibernate Spatial.
- PostgreSQL con PostGIS y JTS.
- MapLibre GL en el cliente.

11.3 Funcionamiento

El catálogo permite consultar puentes y su ubicación en mapa. Administración puede registrar o modificar fichas; un catedrático puede proponer un alta, que queda pendiente de decisión administrativa. Al guardar una ubicación se validan coordenadas dentro de Guatemala y su pertenencia al municipio seleccionado. Las consultas espaciales detectan puentes cercanos para advertir de posibles duplicados; la proximidad no bloquea automáticamente el alta y puede requerir confirmación.

Las asignaciones vinculan puentes activos con inscripciones académicas vigentes. El sistema impide duplicar una asignación activa y registra la revocación con su motivo. La selección disponible para trabajo de campo se deriva de estas asignaciones y del perfil del usuario.

11.4 Diagrama Mermaid



Consideraciones técnicas

- La ubicación se almacena como dato geográfico y se valida espacialmente contra límites municipales; no basta con validar latitud y longitud.
- La alerta de cercanía sirve para revisión de duplicados, no sustituye el criterio de aprobación.
- La revocación afecta la disponibilidad de una asignación para nuevas operaciones; el historial asociado al puente conserva su contexto.

12 Formularios y calibración

12.1 Descripción

Este módulo define versiones de formularios técnicos y sus reglas de evaluación. El cliente construye la captura a partir del esquema publicado, mientras la configuración del IC se mantiene asociada a la versión.

12.2 Tecnologías

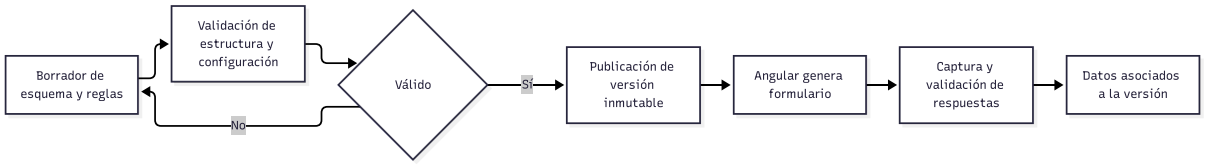
- JSON Schema y validador JSON Schema en Spring.
- Angular Reactive Forms y RxJS.
- PostgreSQL con almacenamiento JSONB.

12.3 Funcionamiento

Una versión de formulario se edita como borrador, se valida y luego se publica. El cliente resuelve el esquema y genera controles reactivos, incluidos campos repetibles y reglas de visibilidad condicional. Al guardar, serializa las respuestas visibles y aplica las validaciones derivadas del esquema.

La configuración de evaluación asociada incluye catálogo de elementos, mapeos entre versiones, escala, pesos y reglas críticas. Antes de publicar, el backend verifica que el esquema y la configuración sean coherentes y que los mapeos requeridos estén completos. La publicación calcula una huella del esquema y fija la versión activa; las inspecciones conservan referencia a la versión con la que fueron iniciadas. La calibración aprobada queda asociada a la versión y conserva su acta.

12.4 Diagrama Mermaid



Consideraciones técnicas

- El esquema de cada versión publicada no se modifica; permite interpretar inspecciones históricas con la estructura original.
- Formularios y reglas se versionan juntos para preservar la relación entre captura y cálculo. El algoritmo del IC se detalla en su propio módulo.
- Las referencias internas del JSON Schema se resuelven con límites de profundidad para evitar recursión indefinida.

- La aprobación de calibración es un proceso separado de la edición del borrador y conserva evidencia documental.

13 Índice de condición

13.1 Descripción

El módulo calcula el índice de condición (IC) de una inspección y, al publicarla, registra el resultado definitivo y actualiza el estado vigente del puente.

13.2 Tecnologías

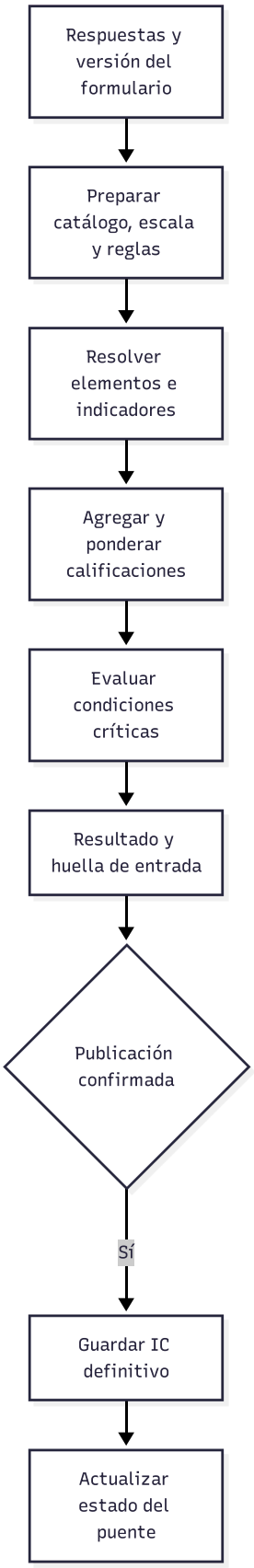
- Spring Boot, transacciones Spring y Jackson.
- PostgreSQL, JPA y consultas SQL para bloqueo transaccional.
- JSON Schema versionado y datos JSON de inspección.

13.3 Funcionamiento

El cálculo toma los datos de inspección junto con el catálogo, los pesos, la escala y las reglas vinculadas a la versión del formulario. Resuelve los elementos evaluables y sus repeticiones, convierte las respuestas a calificaciones, agrega indicadores y aplica las reglas de ponderación y las condiciones críticas. Los elementos excluidos o marcados como no aplicables no participan; los datos requeridos ausentes generan un error de entrada incompleta. El resultado incluye desglose, estado calculado, versión del algoritmo y huella de los datos usados.

Antes de publicar, el sistema vuelve a preparar y calcular el IC definitivo con los datos persistidos. La confirmación permite aceptar el estado calculado o registrar un ajuste justificado cuando las reglas lo permiten. En una sola transacción se guarda el resultado de publicación y se actualiza el estado vigente del puente; un fallo revierte ambos cambios. El recálculo histórico conserva comparaciones y no reemplaza por sí mismo la publicación vigente.

13.4 Diagrama Mermaid



Consideraciones técnicas

- La configuración asociada a la inspección determina cómo interpretar sus respuestas; cambios en versiones posteriores no alteran el cálculo histórico.
- El valor preliminar sirve para revisión de captura; el definitivo se genera al publicar y se almacena junto con el desglose y la huella.
- El bloqueo del puente evita que publicaciones concurrentes compitan por actualizar su estado vigente.
- Los detalles de estados del ciclo de inspección y de revisión se describen en sus módulos respectivos.

14 Archivos y evidencias

14.1 Descripción

Este módulo procesa, almacena y vincula fotografías y documentos con inspecciones, calibraciones y órdenes de mantenimiento.

14.2 Tecnologías

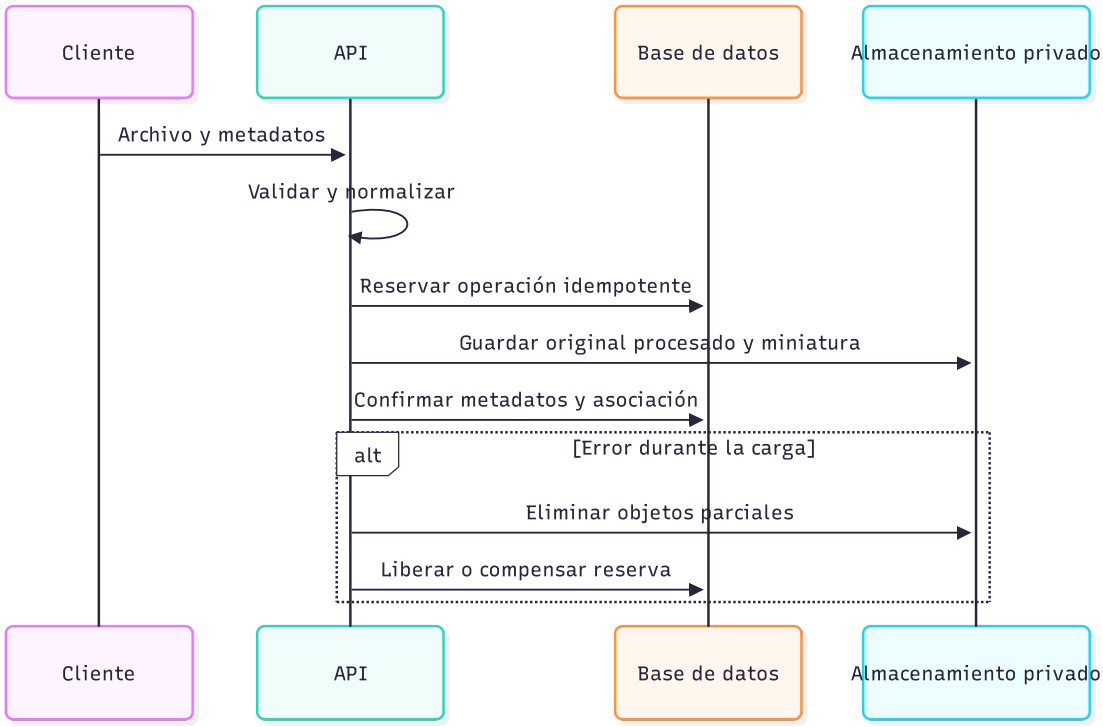
- ImageIO y TwelveMonkeys ImageIO WebP.
- AWS SDK para S3 y almacenamiento local compatible con el proveedor configurado.
- Apache PDFBox para documentos PDF.
- PostgreSQL para metadatos y relaciones de archivos.

14.3 Funcionamiento

Las fotografías se decodifican y normalizan a JPEG RGB; se genera una miniatura y un checksum. La imagen normalizada elimina metadatos EXIF del archivo almacenado, mientras los metadatos de captura que el flujo reconoce se registran por separado. El servicio guarda los objetos bajo claves privadas y asocia sus metadatos al dominio correspondiente. En inspecciones puede conservarse el anclaje a un elemento, instancia y categoría de evidencia.

La carga reserva primero una operación idempotente, almacena los objetos y luego confirma el registro del archivo y su asociación en la transacción del dominio. Si falla una etapa, intenta eliminar los objetos parciales y compensar la reserva. Los documentos se validan de acuerdo con su tipo de uso y el acceso de descarga se somete a autorización. El proveedor de objetos puede ser local o S3/MinIO según la configuración.

14.4 Diagrama Mermaid



Consideraciones técnicas

- Se valida el contenido decodificado, no solo la extensión o el MIME declarado; la normalización limita el lado mayor de la imagen a 1600 px y genera miniatura de 300 px.
- Base de datos y almacenamiento de objetos no comparten una transacción distribuida. Las reservas, compensaciones y procesos de recuperación reducen el riesgo de objetos huérfanos.
- Las claves de almacenamiento no deben tratarse como autorización: el servidor controla el acceso a las descargas.
- Los capítulos de inspección y mantenimiento describen los requisitos de evidencia de cada flujo; aquí se explica el mecanismo común de archivos.

15 Inspecciones

15.1 Descripción

Este módulo coordina el ciclo de vida de una inspección: inicio, captura y guardado de respuestas, envío, publicación directa o revisión académica.

15.2 Tecnologías

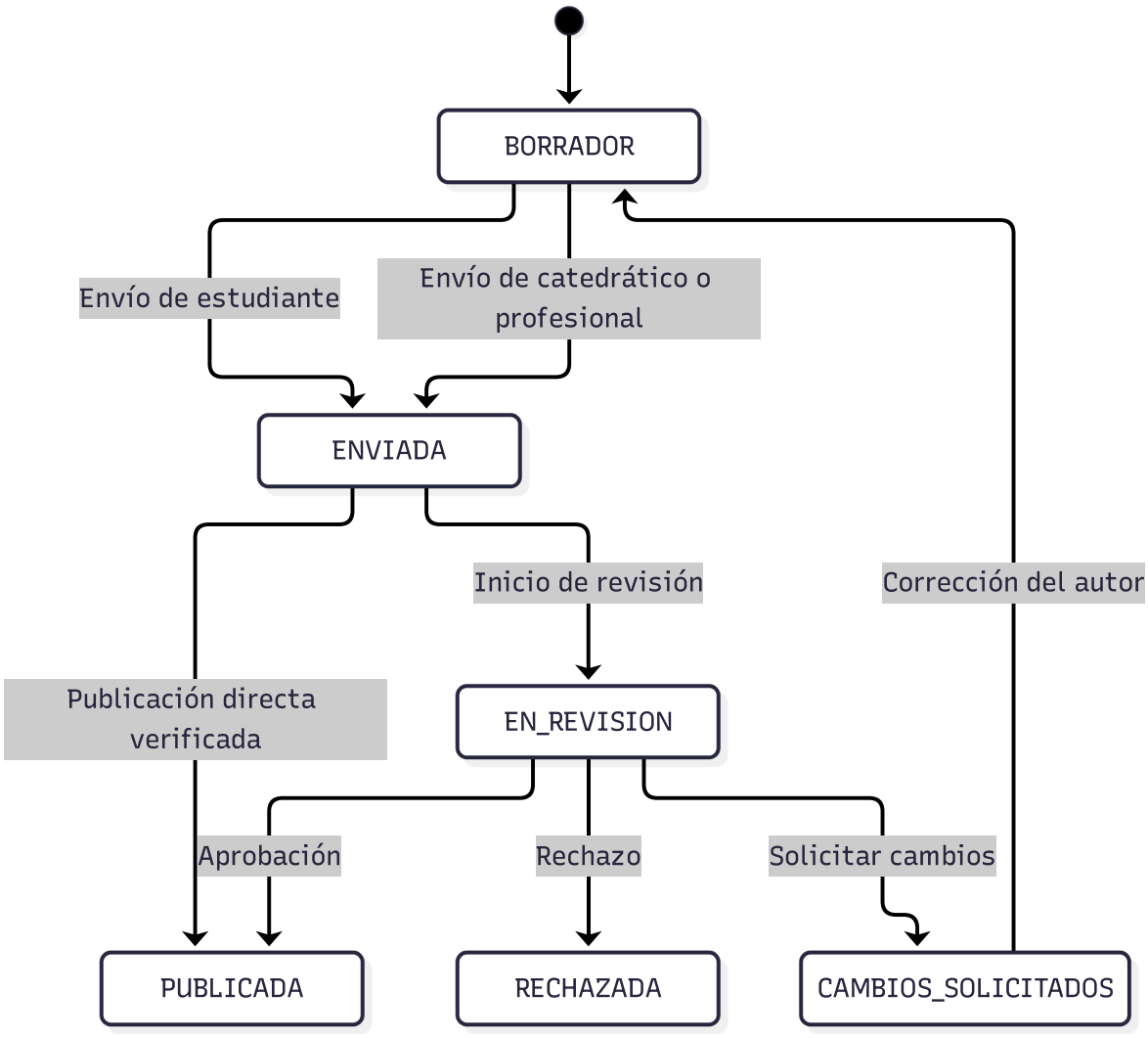
- Spring Boot, Spring Data JPA y Bean Validation.
- Angular y formularios reactivos generados desde esquemas.
- PostgreSQL con JSONB y control transaccional.

15.3 Funcionamiento

Al iniciar, la inspección queda asociada a un puente, a su autor y a una versión concreta del formulario. La captura puede guardarse como borrador y se valida contra esa versión. Antes del envío o publicación se comprueban la completitud de respuestas y las evidencias requeridas para el flujo.

Las transiciones se validan como eventos desde un estado actual y se registran en el historial. Las inspecciones de estudiantes se envían a un ciclo de revisión de un catedrático; las de catedráticos y profesionales externos verificados pueden publicarse directamente. El revisor puede aprobar, solicitar cambios o rechazar, con motivo escrito para las dos últimas decisiones. Una corrección vuelve al autor para edición según el estado permitido. La publicación integra el IC definitivo en la transacción y emite un evento para procesos posteriores.

15.4 Diagrama Mermaid



Consideraciones técnicas

- La máquina de estados rechaza eventos incompatibles y los estados terminales no admiten

nuevas transiciones.
- La concurrencia se controla al modificar la inspección; una solicitud incompatible puede devolver conflicto en lugar de sobrescribir cambios.
- Publicación, historial, IC definitivo y estado vigente del puente se confirman juntos. Las notificaciones y generación de mantenimiento ocurren en etapas posteriores.
- La sincronización offline y los detalles del cálculo, revisión y almacenamiento de evidencias se explican en sus capítulos propios.

16 Offline y sincronización

16.1 Descripción

Este módulo permite preparar salidas de campo, conservar inspecciones y fotografías en el dispositivo y sincronizar las operaciones cuando vuelve la conectividad.

16.2 Tecnologías

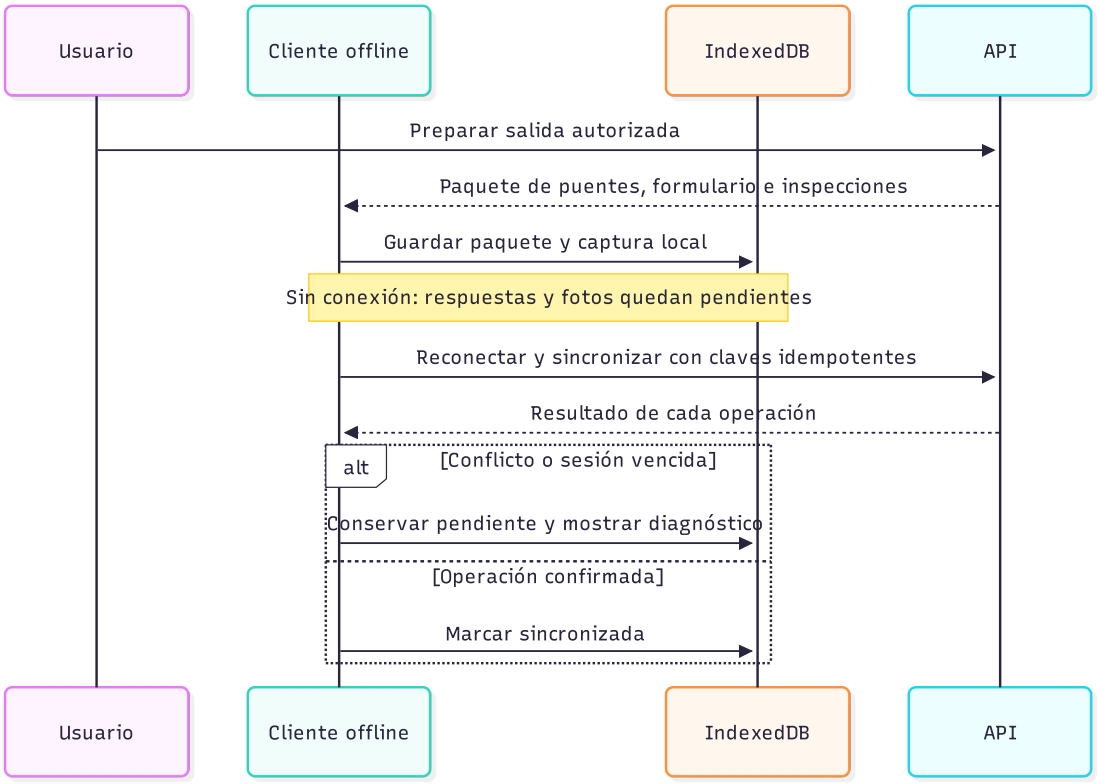
- Angular Service Worker (PWA) y Dexie sobre IndexedDB.
- API REST de sincronización e idempotencia en Spring.
- Almacenamiento del navegador y señales de conectividad.

16.3 Funcionamiento

La API prepara un paquete de trabajo según el usuario, los puentes autorizados y la versión activa del formulario. El cliente guarda el paquete y los datos de campo localmente mediante la capa offline. IndexedDB conserva entidades pendientes y blobs de fotografía; el service worker administra la caché de recursos de la aplicación.

Cuando hay conexión, el sincronizador procesa las operaciones pendientes en orden: crea o actualiza el borrador, carga las fotografías y luego envía o publica la inspección. Cada operación conserva una clave idempotente para repetir solicitudes sin duplicar el efecto. La API vincula la clave con el usuario, método, ruta y huella de solicitud; si recibe la misma operación completada reproduce su respuesta, y rechaza reutilizarla con contenido distinto. Los conflictos de concurrencia quedan identificados para diagnóstico y resolución.

16.4 Diagrama Mermaid



Consideraciones técnicas

- Los datos locales se separan por propietario; cambiar de usuario no debe sincronizar pendientes de otra cuenta.
- Reintentar con la misma clave es seguro solo si se conserva la misma solicitud. La idempotencia no resuelve conflictos de edición concurrente.
- La cuota disponible depende del navegador y dispositivo; la estimación de almacenamiento no garantiza que futuras escrituras tengan éxito.
- El service worker cachea recursos de la app; no reemplaza IndexedDB como persistencia de los registros de campo.

17 Revisión académica

17.1 Descripción

Este módulo gestiona la revisión de inspecciones estudiantiles, sus ciclos, observaciones y resolución por parte del catedrático.

17.2 Tecnologías

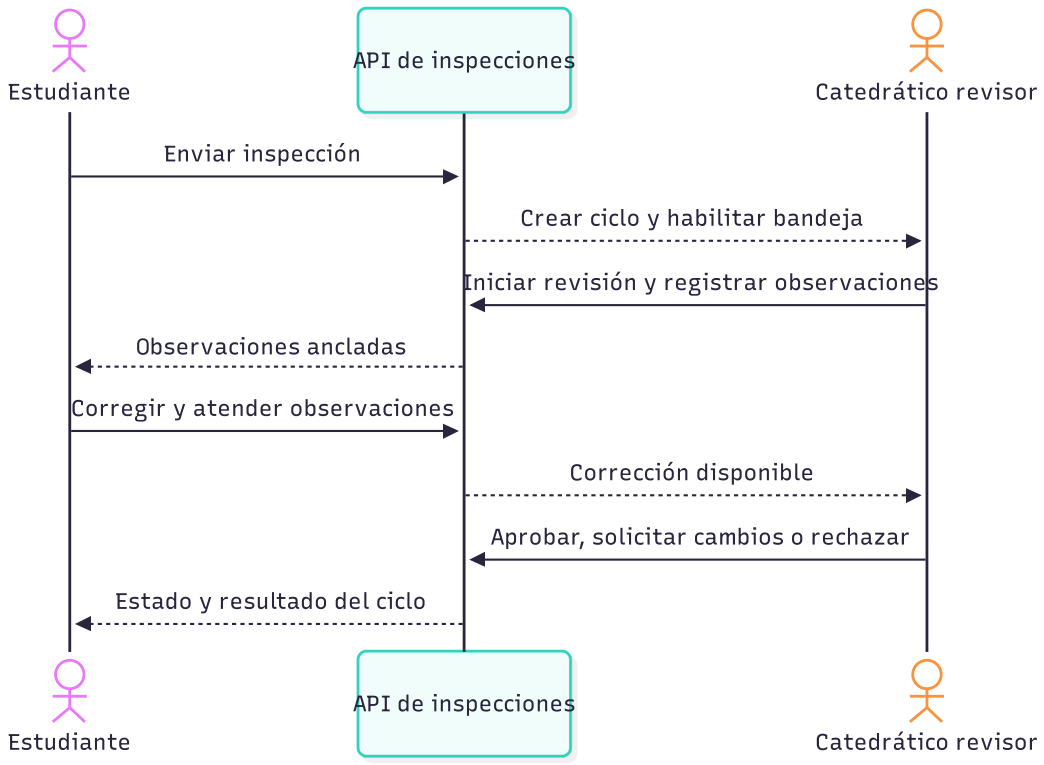
- Spring Boot, Spring Data JPA y transacciones.
- Angular para bandejas, detalle y atención de observaciones.
- PostgreSQL para ciclos, observaciones e historial.

17.3 Funcionamiento

Al enviar una inspección estudiantil se crea un ciclo de revisión asignado al catedrático correspondiente. El revisor consulta la bandeja, inicia el ciclo y revisa respuestas y evidencias. Puede anotar un elemento o instancia del formulario, una categoría fotográfica o una observación general. El backend valida que los anclajes existan en la versión del formulario y que la categoría corresponda al contexto.

El estudiante atiende las observaciones y remite la corrección; el revisor verifica su resolución y emite el veredicto del ciclo. La aprobación publica la inspección y coordina el cálculo definitivo del IC; solicitar cambios devuelve la inspección al autor y el rechazo la cierra. Los ciclos y estados se mantienen para conservar la trazabilidad de revisiones sucesivas.

17.4 Diagrama Mermaid



Consideraciones técnicas

- Solo el catedrático asignado puede resolver su ciclo; el autor y revisor deben ser distintos.
- La revisión se vincula a una versión concreta del formulario, por lo que los anclajes no dependen de la versión activa más reciente.
- La aprobación sigue las validaciones de completitud y evidencia de publicación; el detalle del IC definitivo está en su módulo.
- Las observaciones y correcciones se conservan por ciclo, evitando que una nueva revisión sobrescriba la anterior.

18 Mantenimiento

18.1 Descripción

Este módulo genera propuestas de mantenimiento desde inspecciones publicadas, administra planes rutinarios y controla órdenes hasta su ejecución.

18.2 Tecnologías

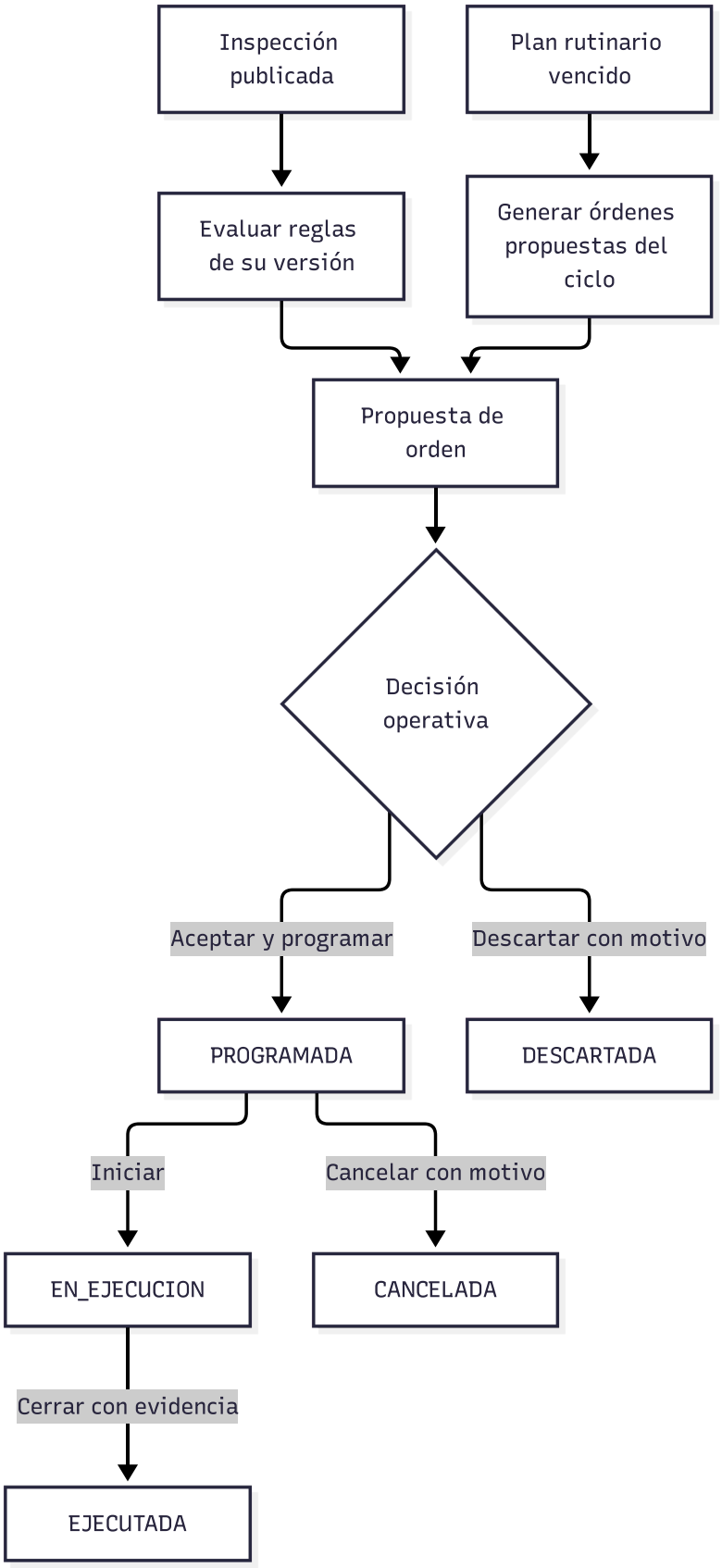
- Spring Boot, Spring Data JPA y eventos transaccionales.
- PostgreSQL y JSONB para reglas y resultados.
- ShedLock para coordinar tareas programadas.

18.3 Funcionamiento

Después de confirmarse la publicación de una inspección, un listener evalúa las reglas activas de la versión del formulario usada. Las condiciones se contrastan con los elementos inspeccionados y el resultado del IC; cada coincidencia puede generar una orden en estado `PROPUESTA`. La generación es idempotente y evita volver a proponer una orden equivalente que siga en curso. Administración puede regenerar propuestas faltantes.

Las propuestas no se aceptan ni programan automáticamente. Un usuario con permisos puede aceptarlas y programarlas, descartarlas con motivo, iniciar su ejecución o cancelarlas con motivo. El cierre como ejecutada exige datos de ejecución y evidencia. En paralelo, los planes rutinarios generan ciclos vencidos y órdenes asociadas; una tarea programada coordina ese trabajo con bloqueo distribuido.

18.4 Diagrama Mermaid



Consideraciones técnicas

- La generación automática ocurre después del commit de la publicación y en una transacción independiente: un error de mantenimiento no revierte una inspección ya publicada.
- Las órdenes generadas permanecen como propuestas hasta una decisión operativa; la prioridad sugerida y la evaluación de reglas no equivalen a programación.
- Los estados terminales no admiten nuevas transiciones; cambios de estado conservan historial.
- Las tareas programadas usan ShedLock para evitar ejecución concurrente entre instancias.

19 Foro y notificaciones

19.1 Descripción

Este módulo habilita la colaboración en torno a puentes e inspecciones publicadas y entrega avisos internos sobre eventos relevantes del sistema.

19.2 Tecnologías

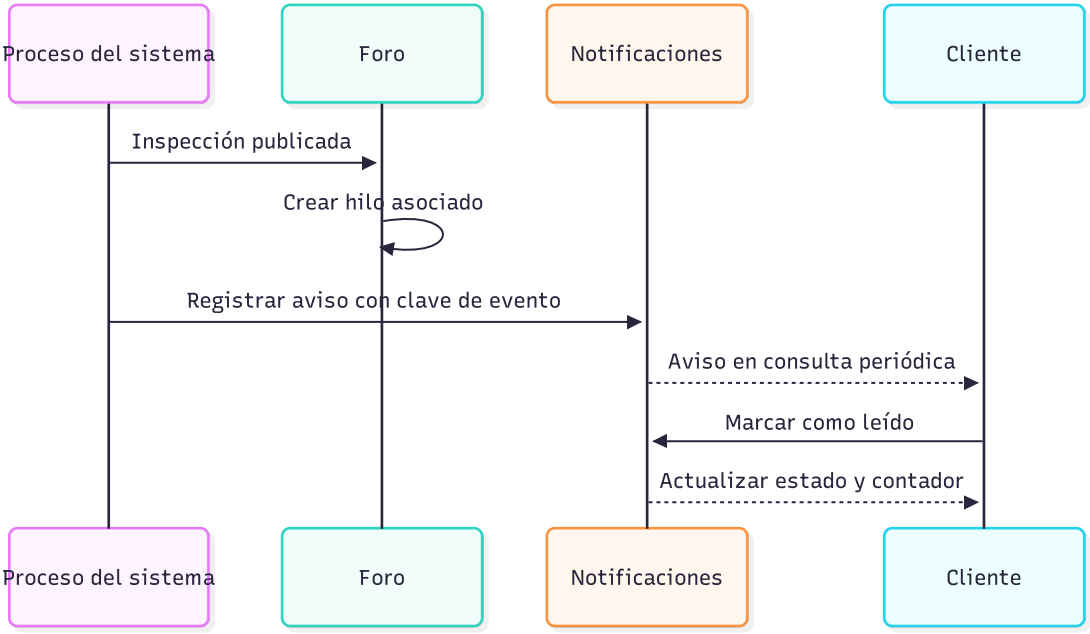
- Spring Boot, Spring Data JPA y eventos transaccionales.
- PostgreSQL para hilos, comentarios y notificaciones.
- Angular para consulta, contadores y avisos offline.

19.3 Funcionamiento

El foro organiza hilos permanentes por puente y un hilo asociado a cada inspección publicada. Los comentarios permiten respuestas de un nivel; en el hilo de inspección pueden anclarse a un campo o elemento del formulario. La lectura es pública en los hilos disponibles, mientras que participar requiere sesión. Catedráticos y administradores pueden moderar comentarios y cerrar hilos; se aplican límites de participación.

La publicación de una inspección crea su hilo después de confirmar la transacción. Los comentarios y otras acciones del sistema pueden producir notificaciones internas para usuarios destinatarios. Cada aviso incluye una clave de evento para evitar duplicados y puede consultarse, marcarse como leído o resumirse por usuario. El cliente actualiza los avisos mediante consultas periódicas y conserva contexto de notificaciones para uso offline.

19.4 Diagrama Mermaid



Consideraciones técnicas

- El hilo asociado a una inspección se habilita solo después de su publicación; su creación ocurre después del commit.
- La validez de un anclaje depende del esquema de la inspección, no del formulario activo actual.
- La clave de evento evita duplicar avisos para el mismo usuario y evento; los textos también se validan para no incluir patrones de credenciales o calificaciones.
- Los avisos de la aplicación son persistentes e internos. Los correos de autenticación e invitación pertenecen a sus flujos respectivos.

20 Auditoría

20.1 Descripción

La auditoría conserva cambios relevantes de negocio con actor, entidad, fecha y valores antes y después, y permite consultar la trazabilidad desde las herramientas administrativas.

20.2 Tecnologías

- Spring AOP y anotaciones de auditoría.
- Spring Transactions y Spring Data JPA.
- Jackson para representar snapshots en JSON.

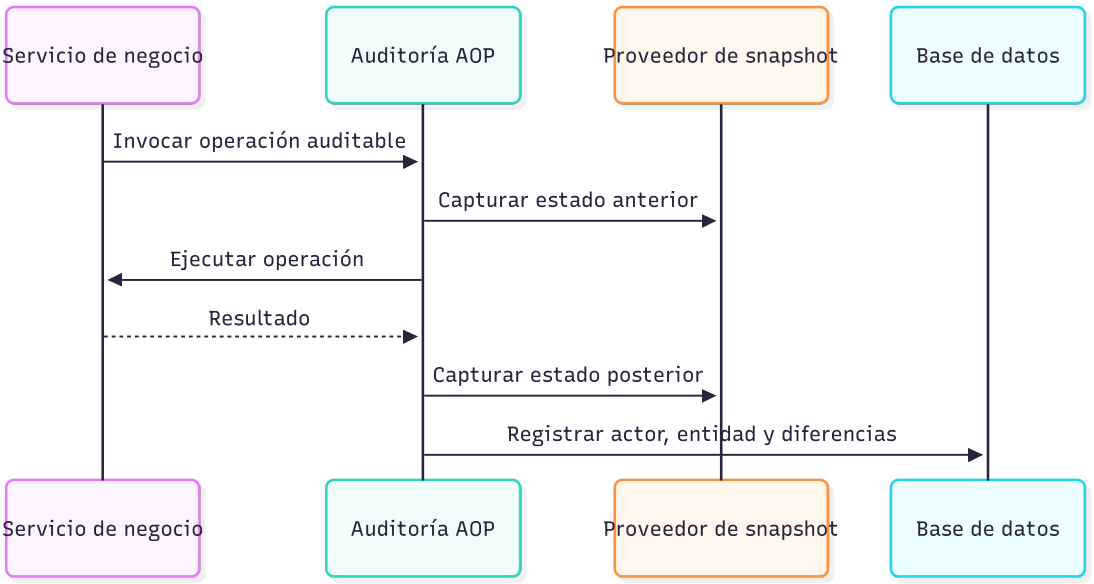
20.3 Funcionamiento

Para operaciones anotadas, un aspecto obtiene el estado anterior mediante proveedores de snapshot, ejecuta la operación y captura el estado posterior. También existen escrituras explícitas

desde servicios cuando una operación requiere registrar auditoría dentro de la misma transacción de negocio. Los procesos programados pueden registrar ejecuciones sin actor humano. La consulta administrativa presenta los registros y sus diferencias.

La escritura puede participar en la transacción existente o ejecutarse en una transacción independiente, según el tipo de registro. Los cambios que requieren atomicidad usan la transacción actual; otros registros se escriben con una frontera transaccional propia. Los logs operativos son un canal distinto y no reemplazan la bitácora persistente.

20.4 Diagrama Mermaid



Consideraciones técnicas

- La captura anterior/posterior depende de proveedores que proyectan datos del dominio; los snapshots no deben exponer secretos o información innecesaria.
- La propagación transaccional define si el registro acompaña al cambio o sobrevive a su rollback. Las llamadas explícitas deben respetar esa intención.
- Las fallas de captura se registran en logs y pueden dejar el snapshot incompleto; los logs no constituyen un reemplazo de la auditoría consultable.
- Los registros de tareas automáticas identifican el proceso y su resumen sin atribuirlo a un usuario.

21 Despliegue y recuperación

21.1 Descripción

Este módulo describe la ejecución del cliente y la API junto con PostgreSQL/PostGIS, el manejo de configuración por entorno, el despliegue automatizado y la recuperación de datos.

21.2 Tecnologías

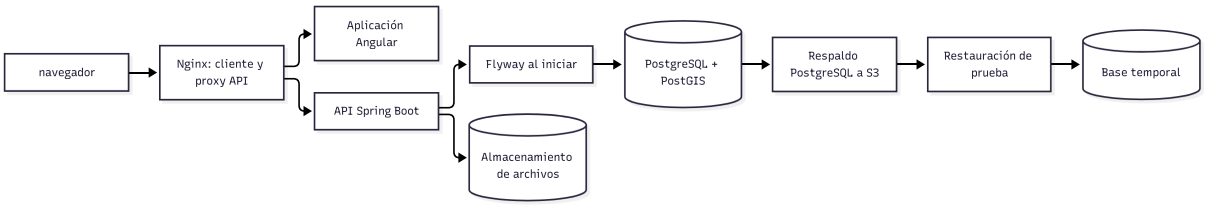
- Java 21, Spring Boot y Maven.
- Angular, Node.js y Nginx.
- Docker Compose, PostgreSQL con PostGIS y Flyway.
- AWS S3 para objetos y respaldos en los perfiles de despliegue que lo configuran.

21.3 Funcionamiento

Las imágenes de producción se construyen por separado para API y cliente. Nginx sirve la aplicación web y enruta las solicitudes API al backend; la API se conecta a PostgreSQL y al almacenamiento de archivos. Las variables y secretos se inyectan en ejecución. Flyway aplica las migraciones al iniciar la API mediante una cuenta de migración distinta de la cuenta de aplicación.

Los scripts de despliegue de staging y producción validan las imágenes candidatas y esperan health checks. El flujo de producción conserva la configuración anterior y puede restaurarla automáticamente si el despliegue falla. Los scripts de respaldo generan volcados de PostgreSQL y los almacenan en S3; el procedimiento de prueba restaura un respaldo en una base temporal y consulta migraciones y datos para comprobarlo.

21.4 Diagrama Mermaid



Consideraciones técnicas

- Los perfiles de ejecución requieren configurar URL y cuentas de base de datos, clave JWT, CORS, correo y almacenamiento; los valores de ejemplo no son secretos válidos.
- La cuenta de migración requiere permisos de esquema distintos de la cuenta de aplicación. Las migraciones aplicadas son parte del historial operativo.
- El rollback del despliegue restaura imágenes y composición previa; no revierte cambios de esquema de base de datos.
- La existencia de scripts de AWS, S3 o staging describe capacidad configurada en el proyecto y no demuestra que un ambiente esté desplegado o que una restauración haya sido ejecutada.
- La recuperación debe validarse periódicamente con restauraciones de prueba, no solo con la existencia de un archivo de respaldo.